Introduction à la Programmation et l'Algorithmique

CHRISTELLE CAILLOUET – CHRISTELLE.CAILLOUET@UNIV-COTEDAZUR.FR

DINO LOPEZ PACHECO – DINO.LOPEZ@UNIV-COTEDAZUR.FR

Foreword

- The content of this slide has been strongly inspired from the following materials:
  - "Programmation et Systèmes" lecture, SI3 at Polytech, written by Stephane Lavirotte et Erick Gallesio
  - "Practical Programming in C", from the MITOPENCOURSEWARE - https://ocw.mit.edu/courses/6-087-practical-programming-in-c-january-iap-2010/
- we advise the reading of the
  - textbook "An Introduction to the C Programming Language and Software Design" written by Tim Bailey - https://www-personal.acfr.usyd.edu.au/tbailey/ctext/ctext.pdf
  - And any other book or Internet tutorials you might find

Introduction to C programming

Why C Programming

- In this lecture, we want to explore the properties of
  - Data structures
  - Some frequently employed algorithms
- Performance of data structures and algorithms are related to their internal architecture
  - Easily "visible" when writing C code
- C is a language
  - Close to the processor semantics
  - Providing an easy access to low level system mechanisms
  - Easy and effective
  - One with the highest memory and processing efficiency
  - The language to write many Operating Systems, drivers and compilers
  - But… low-level! -> big power, big responsibility!
- Linus Torvalds said (2012)
  - "If you think like a computer, writing C actually makes sense"

Structure of a C program

1. Documentation
2. Link (useful functions -libraries- for our program)
3. Definition (macros + constants)
4. Global declarations (variables + functions)
5. The “main()” function
6. Subprograms

```
/************************************************
 * Program that welcomes you to the IPA course
 * author: me
 * date: sept 1st, 2023
 ************************************************/
```

Structure of a C program

1. Documentation
2. Link (useful functions -libraries- for our program)
3. Definition (macros + constants)
4. Global declarations (variables + functions)
5. The “main()” function
6. Subprograms

```
#include <stdio.h>
#include <string.h>
```

Structure of a C program

1. Documentation
2. Link (useful functions -libraries- for our program)
3. Definition (macros + constants)
4. Global declarations (variables + functions)
5. The "main()" function
6. Subprograms

```
#define MAXVAL 1000
```

- All lines starting with "#" are preprocessor directives
- A preprocessor process the code before it is actually compiled

Structure of a C program

1. Documentation
2. Link (useful functions -libraries- for our program)
3. Definition (macros + constants)
4. Global declarations (variables + functions)
5. The "main()" function
6. Subprograms

```
int currentval = 50;

void say_welcome(void);
```

Structure of a C program

1. Documentation
2. Link (useful functions -libraries- for our program)
3. Definition (macros + constants)
4. Global declarations (variables + functions)
5. The "main()" function
6. Subprograms

```
int main(int argc, char *argv[]) {
    int x;
    say_welcome();
    x = 10;
    x += currentval;
    printf("the current value of x is %d \n", x);
}
```

Structure of a C program

1. Documentation
2. Link (useful functions -libraries- for our program)
3. Definition (macros + constants)
4. Global declarations (variables + functions)
5. The “main()” function
6. Subprograms

```
void say_welcome(void) {
    printf(“welcome to the IPA lecture\n”);
}
```

All at once

```
/*************************************************
 * Program that welcomes you to the IPA course
 * author: me
 * date: sept 1st, 2023
 *************************************************/

#include <stdio.h>

#define MAXVAL 1000

int currentval = 50;

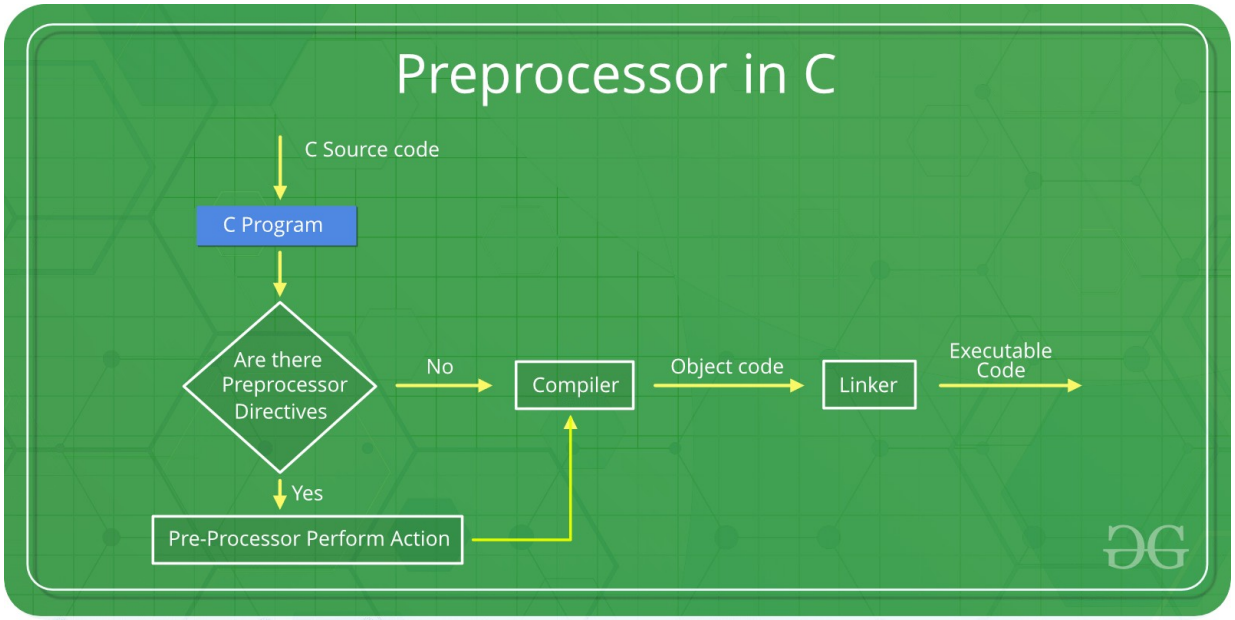
void say_welcome(void);

int main(int argc, char *argv[]) {
   int x;
   say_welcome();
   x = 10;
   x += currentval;
   printf("the current value of x is %d \n", x);
}

void say_welcome(void) {
   printf("welcome to the IPA lecture\n");
}
```

Compiling a C program

- With GCC
  - gcc -o welcome welcome.c
- To execute your program
  - Suppose it's not in the PATH
  - ./welcome



From https://www.geeksforgeeks.org/cc-preprocessors/

Numeric and char data types

- C is a *weak* typed language
  - Allows implicit conversions
  - Forced (casting) conversions
- Numeric data types:
  - short (or short int), int, long, float, double, char
  - By default, all data types store signed values (e.g. -1)
    - One bit is used to do so
  - short, int, long and char can be prefixed with "unsigned", so all bits are used to write a number
- Sizes of data types depends on the architecture of the system/compiler
- However, the following should hold
  - sizeof(char) < sizeof(short)<= sizeof(int) <= sizeof(long)
  - sizeof(char) < sizeof(short)<= sizeof(float) <= sizeof(double)

Blocks

- When writing C code, we will frequently build block of codes
  - Use curly braces
  - Replacement for one single statement
- A block is compiled as a single unit
- Variables can be be declared inside a block
  - Scope is limited to that block
  - Nested blocks
- Empty block: {}

```
{
 int i = 2;
 {
  int j = 2;
  int a = i * j;
 }
 // j doesn't exist here
}
```

User defined data types: structure and union

- Combine different elements in one single data type
- Size of the structure is equal to the sum of its members
- Syntax:

```
struct [structure name]
  {
     member definition;
     member definition;
     ...
     member definition;
  } [variable(s)_name(s)];
```

- Store different data types in one memory location
- Size of the union equal to the size of the largest members
- Syntax:

```
union [union name]
  {
     member definition;
     member definition;
     ...
     member definition;
  } [variable(s)_name(s)];
```

User defined data types: structure and union

Access struct and union members with the "." (member operator) or "->" (structure pointer operator)

```
struct person {
        int age;
   int salary;
} employee1;


struct person employee2;
employee2.age = 30;
employee2.salary = 2500;
```

Memory size of the structure is the sum of the memory region taken by every element (each element has its own memory region)

```
union clientaddr {
   char postaddr[200];
   int boxnumber;
} c1;


union clientaddr c2;
c1.boxnumber = 3456;
strcpy(c2.postaddr,"anystreet #10, 01234, CA\0");
```

Memory size of the union is equal to the size of its largest element (all elements share the same memory region)

Constants

- Fixed values or literals that can be assigned to variables or be used in some expressions
  - long: 50L
  - unsigned long: 50UL
  - long long int: 50LL
  - int: 50 (decimal), 0x5F (hexadecimal), 010 (octal)
  - float: 50F
  - double: 33.3333333333L
  - char: 'B', '\x42', '\0120'
  - String (not a data type!): "welcome to IPA\0"
- Enumeration:
  - enum WEEK {MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY}
    - MONDAY=0, TUESDAY=1,…

From https://www.examveda.com/images/solution-image/c-program/operators-and-expressions/1493036926-c-programming-c-operator-precedence-table.png

examveda.com

C Operator Precedence Table

This page lists C operators in order of *precedence* (highest to lowest). Their *associativity* indicates in what order operators of equal precedence in an expression are applied.

Operator	Description	Associativity
( ) [ ] . -> ++ --	Parentheses (function call) (see Note 1) Brackets (array subscript) Member selection via object name Member selection via pointer Postfix increment/decrement (see Note 2)	left-to-right
++ -- + - ! ~ (*type*) * & sizeof	Prefix increment/decrement Unary plus/minus Logical negation/bitwise complement Cast (convert value to temporary value of *type*) Dereference Address (of operand) Determine size in bytes on this implementation	right-to-left
* / %	Multiplication/division/modulus	left-to-right
+ -	Addition/subtraction	left-to-right
<< >>	Bitwise shift left, Bitwise shift right	left-to-right
< <= > >=	Relational less than/less than or equal to Relational greater than/greater than or equal to	left-to-right
== !=	Relational is equal to/is not equal to	left-to-right
&	Bitwise AND	left-to-right
^	Bitwise exclusive OR	left-to-right
|	Bitwise inclusive OR	left-to-right
&&	Logical AND	left-to-right
||	Logical OR	left-to-right
? :	Ternary conditional	right-to-left
= += -= *= /= %= &= ^= |= <<= >>=	Assignment Addition/subtraction assignment Multiplication/division assignment Modulus/bitwise AND assignment Bitwise exclusive/inclusive OR assignment Bitwise shift left/right assignment	right-to-left
,	Comma (separate expressions)	left-to-right

Note 1:
Parentheses are also used to group sub-expressions to force a different precedence; such parenthetical expressions can be nested and are evaluated from inner to outer.

Note 2:
Postfix increment/decrement have high precedence, but the actual increment or decrement of the operand is delayed (to be accomplished sometime before the statement completes execution). So in the statement **y = x * z++;** the current value of **z** is used to evaluate the expression (*i.e.,* **z++** evaluates to **z**) and **z** only incremented after all else is done.

www.examveda.com

Functions

Declaration

- Function prototype
  - int sum(int a, int b);
- Syntax
  - return_type function_name(args);
  - return_type or “void”
  - Args or “void”

Definition

- To return a value from the function
  - return expression;
- Use “return;” in a void type function

```
return_type function_name(args) {
    variable declaration;
    program statements;
}
```

If statement

```
if (cond)
   expression;
[else
   expression;]
```

```
if (a < b) {
   x = a;
} else {
   x = b;
}
```

- Use {} in case of multiple expressions
- *Syntactic sugar* with a ternary operator
  - x = a < b ? a : b;

Switch statement

```
switch (expression)
{
   case constant1:
    // statements
    break;
   case constant2:
    // statements
    break;
   .
   .
   default:
    // default statements
}
```

```
switch(val)
{
  case ‘a':
     enable_all_argument();
     break;
  case ‘h':
     show_help();
     break;
  case ‘r':
     reverse_option();
     break;
  default:
     printf("Error! invalid argument");
  }
```

In case of a match, execute the code until a “break;” is reached

Loops: while, do while and for

```
while(condition) {
   statement(s);
}

do {
   statement(s);
} while( condition );

for ( init; condition; increment ) {
   statement(s);
}
```

```
i = 0;
while (i < 10) {
   printf("%d\n", i);
   i += 2;
}

i = 0;
do {
   printf("%d\n", i);
   i += 2;
} while (i < 10);

for( i = 0; i < 10; i += 2 ){
   printf("%d\n", i);
}
```

Arrays

- Arrays is a basic data structure
- Arrays have a fixed length
- You need to
  - Provide a length when declaring an array or
  - Declare and initialize it, so the compiler can determine the length of the array
- First index is 0, and last, n-1
- In multidimensional arrays, you can omit the length of the first set only

```
int numbers[10];

int numbers = {0, 4, 15, 26}; // length is 4

int morenumbers[10][3];

int nums[][5] = {{0,2,4,6},{1,3,5,7,9}};

printf("%d", nums[0][3]); // will print …

char str[] = "you\0";

char str[] = "you”; // the compiler will add ‘\0’ at the end

int func(char var[]);
```

printf()

- Write to stdout
  - int printf(const char *format,…)
  - forrmat: string of characters to be written to stdout
  - Return the number of written characters or a negative number in case of error
- format: constant characters and tags
  - Tags:
    %[flags][width][.precision][length]specifier
  - (some) specifiers: d (integer), u (unsigned int), c (character), f (floating point), o (signed octal), x (unsigned hexadecimal), s (string of characters)

- printf("character is %c\n", car);
- printf("integer number: %d\n", val);
- printf("a double number: %f", dnum);
- printf("hello %s", str);
- printf("%d square is equal to %d", var, var*var);

Which library to include in our program

- C libraries provide multiple functions that we can use in our programs
  - Example: strcpy() to copy one string into another memory region
- So which library should I include to use strcpy() ?
  - Read the man page!
  - $ man strcpy
  - You"ll also get the function definition and know how to use the function

- Another example

$ man strlen

strlen - calculate the length of a string

SYNOPSIS

```
#include <string.h>

size_t strlen(const char *s);
```

DESCRIPTION

The strlen() function calculates the length of the string pointed to by s, excluding the terminating null byte ('\0').